BOILERROOM PROJEKT

David Lindblom & Veronica Czarnotta

Live Demo: kottekliv.com
Github Repo: https://github.com/goteborg-1/kottekliv-productivity-app

1. Sammanfattning

Kottekliv är en produktivitetsapplikation byggd i React (TSX) med fokus på tidshantering och visualisering av arbete. Appen kombinerar en avancerad timer (Stopwatch/Pomodoro) med uppgiftshantering och statistik.

2. Teknisk Stack (Verktyg):

- **Arkitektur:** Single Page Application (SPA) samt mobilkomponenter i React Native..
- **Kodstruktur:** Användning av en shared-mapp (monorepo) för att dela återanvändbara komponenter, typer och logik mellan webb- och mobilprojektet.
- **State Management:** useContext, useReducer och React States.
- **Språk:** Fullständig migrering från JS/JSX till TypeScript (TS/TSX).
- **Routing:** react-router-dom (implementerat även för uppgiftslistor).
- **Datavisualisering:** Recharts för grafisk representation av produktivitetsdata.
- **Komponenter & Design:** lucide-react och react-icons.
- **API-hantering:** Axios för strukturerad datakommunikation.
- **Logik:** Custom hooks för tidsformatering (useFormatTime).
- **Testning:** Mycket manuell acceptanstestning kompletterat med enhetstester i Jest.
- **Infrastruktur:** Cloudflare Pages (Hosting), GitHub & GitHub Kanban (Projektplanering).
- **Tillgänglighet:** Anpassad för tillgänglighet genom tangentbordsnavigering och kontrollerade färgkontraster.

Funktionalitet:

- **Multi-plattform (Webb & Mobil):** Centrala delar som Timer-systemet och Task
- **Datapersistens:** LocalStorage för lagring av användardata i webbläsaren.
- **Task Management:** Multipla uppgiftslistor med synkronisering mellan applikationens olika vyer.
- **Timer-system:** Stopwatch samt Pomodoro-funktionalitet.
- **Sessionshantering:** Möjlighet att addera sessioner manuellt samt redigera/radera existerande historik.
- **Dataanalys:** Dynamiska grafer för visuell analys av tidsanvändning.
- **UX:** Support för Dark och Light mode samt full responsivitet för olika skärmstörlekar.

3. Motgångar och Lärdomar

Resursallokering: På grund av reducerad gruppstorlek (2 personer) tillämpades strikt prioritering. Endast MVP-funktionalitet (Minimum Viable Product) prioriterades i Kanban-flödet.
Refaktorisering: Migrering av hela kodbasen från JS till TS genomfördes halvvägs in i projektet. Innebar även anpassning och harmonisering av komponenter skrivna av tidigare gruppmedlemmar.
Samarbete: Utveckling skedde tvärfunktionellt i varandras kodmoduler. Detta ställde höga krav på kodläsbarhet och informativa kommentarer för effektiv koddokumentation.
Gränssnitt: Standardisering av språkbruk (engelska) mot slutanvändaren för att åtgärda initial inkonsekvens.
Teknisk Precision: Revision av timer-logik. Initial tidmätning via lokala millisekunder ersattes med en lösning baserad på jämförelse mot tids-API för att eliminera tidsdrift.
Arkitektur: Hanteringen av delad kod mellan webb och mobil blev svårare än väntat då logik och plattformsspecifik kod inledningsvis blandades. Vi har påbörjat arbetet med att bryta ut logik till fristående filer i shared-mappen – en viktig insikt kring separation av UI och logik som vi tar med oss till nästa projekt.

4. Intressanta val vi gjort utöver kravlistan

Separation of Concerns (Reducer/Context): Valet att kombinera useReducer med useContext gjordes för att separera komplex affärslogik (paustid, statuskontroll) från UI-komponenterna. Detta ökar testbarhet och skalbarhet.
Centraliserad hjälplogik: Implementering av custom hooks (useFormatTime) och hjälpfunktioner (FormatHelper.ts) för att garantera konsekvent tidsåtergivning (millisekunder till HH:MM:SS) i hela systemet. Den visar dessutom inte “onödig tid” - 00H:33M:05S -> visas bara som “33 min 5s” istället för att ha med timmarna.